ACTÚA COMO ARQUITECTO PRINCIPAL DE TODOENUNO OS. A

Perfecto.

Y te voy a hacer una recomendación de arquitecto antes de escribir una sola línea de SQL:

No construyamos OS Events como una tabla. Construyámoslo como el núcleo de TodoEnUno OS.

Porque si lo hacemos bien:

Código — Texto sin formato

```
Pedidos
Marketing
UniChat
Social Inbox
Proveedores
Logística
Rentabilidad
Finanzas
OS Memory
IA

```

todos dependerán de la misma capa.

VISIÓN FINAL

Hoy tienes:

Código Texto sin formato

```
Módulos aislados

```

Código Texto sin formato

```
Pedido
Proveedor
Anuncio
Conversación
Pago

```

Queremos llegar a:

Código Texto sin formato

```
OS Event Stream

```

Código

Texto sin formato

```
Cliente escribió
↓
Lead generado
↓
Pedido creado
↓
Pedido confirmado
↓
Pedido enviado
↓
Pedido entregado
↓
Dinero cobrado

```

Todo conectado.

ROADMAP OFICIAL DE OS EVENTS

Yo lo dividiría en 8 fases.

FASE 0

Auditoría y Congelación

Objetivo:

No romper nada.

Primero documentamos fuentes.

Sistema actual

Eventos existentes:

Código Texto sin formato

```
whatsapp_orders
gastos_publicidad
proveedores
licitaciones
fb_leads
fb_messages
bb_messages


```

`bb_messages`

Mapeamos:

Código Texto sin formato

```
Tabla
↓
Evento futuro

```

Ejemplo:

Código Texto sin formato

```
whatsapp_orders
↓
order_created

```

Entregable:

Código Texto sin formato

```
docs/os-events-map.md

```

FASE 1

Crear OS Event Store

Primer ladrillo.

Schema nuevo:

Código SQL

```
os

```

Tablas:

Código SQL

```
os.events

```

Código SQL

```
os.event_types
```

```
2
```

Código SQL

```sql
os.event_consumers
```

Nada más.

Todavía no moveremos lógica.

Objetivo:

Tener un lugar donde TodoEnUno registre hechos.

FASE 2

Normalización de Eventos

Definir lenguaje único.

Ejemplo:

NO:

Código Texto sin formato

```
pedido
orden
venta
compra

```

SI:

Código Texto sin formato

```
order.created
order.confirmed
order.shipped
order.delivered

```

Conversaciones:

Código Texto sin formato

```
chat.message.received
chat.message.sent
lead.generated


```

`lead.generated`

Marketing:

Código Texto sin formato

```
marketing.post.created
marketing.lead.generated
marketing.ad.expense


```

`marketing.ad.expense`

Finanzas:

Código Texto sin formato

```
finance.expense.created
finance.income.created
finance.debt.created


```

`finance.debt.created`

FASE 3

Order Event Producer

Primer productor.

Fuente:

Código SQL

```
whatsapp_orders

```

Cuando aparece:

Código Texto sin formato

```
nuevo pedido

```

crear:

Código JSON

```
{
"type":"order.created"
}

```

Cuando cambia estado:

Código Texto sin formato

```
entregado

```

crear:

Código JSON

```
{
"type":"order.delivered"
}

```

Resultado:

Ya existe trazabilidad histórica.

FASE 4

Marketing Event Producer

Fuente:

Código SQL

```
gastos_publicidad

```

Código SQL

```
fb_leads

```

Código SQL

```
post_queue

```

Generar:

Código Texto sin formato

```
marketing.ad.expense
marketing.lead.generated
marketing.post.published


```

`marketing.post.published`

Aquí empieza a nacer rentabilidad.

FASE 5

Supplier Event Producer

Fuente:

Código SQL

```sql
proveedores
licitaciones
licitacion_proveedores


```

`licitacion_proveedores`

Eventos:

Código Texto sin formato

```
supplier.created
supplier.selected
supplier.bid.received
purchase.requested


```

`purchase.requested`

Esto prepara abastecimiento.

FASE 6

Money Engine

Hasta aquí NO hay finanzas.

Luego aparece:

Código Texto sin formato

```
Finance Projector

```

Consume eventos.

Ejemplo:

Código Texto sin formato

```
order.delivered

```

↓

Código Texto sin formato

```
finance.income.created

```

Código Texto sin formato

```
marketing.ad.expense
```

```
2
```

↓

Código Texto sin formato

```
money_transaction

```

Aquí nacen:

Código SQL

```
finance.money_transactions
finance.cash_accounts
finance.daily_metrics


```

`finance.daily_metrics`

Importante:

Finanzas NO produce eventos.

Finanzas consume eventos.

FASE 7

OS Memory

Mi favorita.

Todos los eventos se convierten en memoria.

Ejemplo:

Código JSON

```
{
"date":"2026-11-06",
"orders":42,
"revenue":1250,
"ad_spend":180,
"top_product":"Mini EMS"
}

```

No volveremos a calcular.

OS Memory recuerda.

FASE 8

Intelligence Layer

Finalmente:

Código Texto sin formato

```
Copilot

```

o agentes.

Podrán preguntar:

Código Texto sin formato

```
¿Qué producto dejó más dinero?

```

Código Texto sin formato

```
¿Qué anuncio trae clientes que realmente compran?

```

Código Texto sin formato

```
¿Por qué bajó la utilidad?

```

Porque todo está conectado por eventos.

Arquitectura física recomendada

Supabase:

Código Texto sin formato

```
public

```

↓

actual

Código Texto sin formato

```
os

```

↓

nuevo

Código SQL

```
os.events
os.event_types
os.event_consumers

```

Código Texto sin formato

```
finance

```

↓

más adelante

Código SQL

```
finance.money_transactions
finance.cash_accounts
finance.daily_metrics

```

Código Texto sin formato

```
memory
```

↓

futuro

Código SQL

```sql
memory.facts
memory.daily_snapshots
memory.insights
```

Qué construiría mañana mismo

Sprint 1.

Nada de dashboards.

Nada de finanzas.

Nada de IA.

Solo:

Paso 1

Código SQL

```sql
create schema os;

```

Paso 2

Código SQL

```sql
os.events

```

Paso 3

Código SQL

```sql
os.event_types

```

Paso 4

Código SQL

```sql
trigger:
whatsapp_orders
```

```
→ os.events
```

Paso 5

Página nueva:

Código Texto sin formato

```
Sistema
└─ Event Stream
```

Mostrando:

Código Texto sin formato

```
12:01 order.created
12:02 lead.generated
12:04 chat.message.received
12:08 order.confirmed
12:15 order.delivered
```

`12:15 order.delivered`

Porque en cuanto puedas visualizar el flujo vivo de eventos, el resto de TodoEnUno OS empieza a construirse encima de una única fuente de verdad.

Y sinceramente, viendo tu repositorio y tu base actual, creo que **OS Events es probablemente el ladrillo más importante que**

puedes construir antes de Finanzas, OS Memory y los agentes de IA.